

DSAA3073: Theories in Data Science

Week 6: Optimization & Ensemble Methods

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

Opening Challenge

Opening Challenge

The Weak Learning Puzzle

Suppose you have access to a **weak learning algorithm** (an algorithm that produces classifiers slightly better than random guessing) that, given any distribution over training examples, can produce a classifier with weighted training error at most 49% (just slightly better than random guessing at 50%).

Question: Can you use this weak learner to construct a classifier with arbitrarily small error rate, say 1% or even 0.1%?

Constraints:

- You can only call the weak learner multiple times
- Each call uses a different distribution over the training data
- You must combine the weak classifiers somehow

This seemingly impossible task is the foundation of **boosting theory** (the theory of combining weak classifiers into strong ones).

Introduction

The term “boosting” was coined by Robert Schapire in his 1990 paper, which gave the first theoretical construction showing that weak learning implies strong learning.

Key Takeaway

Conceptual Backbone: From Convex Optimization to Ensemble Learning

This week connects three fundamental ideas:

1. **Convex optimization** (Week 1 foundations): **Gradient descent** (an iterative optimization method) on smooth loss functions
2. **Coordinate descent** (an optimization method that optimizes one variable at a time while holding others fixed): Greedy optimization in high-dimensional spaces
3. **Ensemble learning** (combining multiple models to create a stronger predictor): Combining weak learners through adaptive reweighting

The journey: We start with gradient descent principles, show how **AdaBoost** (Adaptive Boosting, a specific boosting algorithm) performs coordinate descent on **exponential loss** (a smooth convex loss function), prove exponential convergence via convexity, and explain generalization through margin theory.

In the late 1980s, Michael Kearns and Leslie Valiant posed a fundamental question in **computational learning theory** (the theoretical study of machine learning algorithms): **Is weak learning equivalent to strong learning?** In other words, if we can learn slightly better than random guessing, can we learn arbitrarily well?

→ Week 1 Preview: Convexity & Optimization

The exponential convergence we prove in Section 4 directly exploits the **convexity** of exponential loss—a concept introduced in Week 1. The coordinate-descent perspective in Section 6 shows AdaBoost is performing gradient-based optimization.

This question was not merely theoretical curiosity. Many practical learning algorithms produce classifiers that are only moderately accurate. If we could systematically combine many such “weak” classifiers into a highly accurate “strong” classifier, we would have a powerful general-purpose learning technique.

The answer turned out to be a resounding **yes**, leading to one of the most successful families of machine learning algorithms: **boosting algorithms**. The most famous of these, **AdaBoost** (Adaptive Boosting), developed by Yoav Freund and Robert Schapire in 1997, has become a cornerstone of modern machine learning.

What Makes Boosting Special?

Boosting algorithms have several remarkable properties:

1. **Theoretical guarantee:** They provably convert weak learners into strong learners
2. **Practical effectiveness:** They achieve state-of-the-art performance on many tasks
3. **Simplicity:** The core algorithm is surprisingly simple and elegant

4. **Versatility:** They work with any weak learning algorithm as a black box
5. **Margin maximization:** They implicitly maximize classification margins, improving generalization

Connection

Boosting provides a constructive proof that weak PAC learnability implies strong PAC learnability under the usual PAC assumptions. If a concept class is weakly learnable (population error $\leq \frac{1}{2} - \gamma$ for some $\gamma > 0$ under the formal weak-learning oracle), then it is strongly learnable.

In the algorithmic sections below, we first prove an **empirical training-error** guarantee for a fixed sample. Turning that into a PAC population guarantee additionally requires generalization control for the combined classifier.

Learning Objectives

By the end of this week, you will be able to:

1. Formalize the weak learning hypothesis and understand its implications
2. Derive and implement the AdaBoost algorithm from first principles
3. Prove that AdaBoost minimizes exponential loss via coordinate descent
4. Analyze the training error convergence using the weak learning condition
5. Apply margin theory to explain AdaBoost's generalization performance
6. Understand why AdaBoost continues to improve even after training error reaches zero

✓ Checkpoint

Checkpoint 1: Foundations

Before proceeding, ensure you understand:

- Convex functions and gradient descent (Week 1)
- PAC learning framework (Week 2-3)
- Margin-based generalization (Week 6, SVMs)

Self-check: Can you explain why convex loss functions are easier to optimize than 0-1 loss?

Roadmap

Our journey through boosting theory follows this path:

Section 2: The Boosting Problem

- Formalize weak vs. strong learning
- Understand the boosting question
- Preview the solution strategy

Section 3: The AdaBoost Algorithm

- Derive the algorithm step by step
- Understand the weight update mechanism
- Interpret the hypothesis weights

Section 4: Training Error Analysis

- Prove exponential convergence of training error
- Connect to the weak learning condition
- Understand the role of the edge parameter

Section 5: The Margin Explanation

- Define classification margins
- Prove margin-based generalization bounds
- Explain why test error improves after training error = 0

Section 6: Loss Minimization Perspective

- Show AdaBoost minimizes exponential loss
- Interpret as coordinate descent
- Compare with other loss functions

Key Takeaway

Boosting transforms the seemingly impossible task of converting weak learners into strong learners into a practical, efficient algorithm with strong theoretical guarantees.

The Boosting Problem

The edge γ measures how much better than random guessing the weak learner performs. Even $\gamma = 0.01$ (51% accuracy) is sufficient for boosting!

Weak Learning vs. Strong Learning

Let's formalize the distinction between weak and strong learning in the binary classification setting.

Definition: Weak Learner

A **weak learner** (an algorithm that produces classifiers slightly better than random guessing) for a binary classification problem is an algorithm that, given:

- Training set $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$ where $x_i \in \mathcal{X}$, $y_i \in \{-1, +1\}$
- Distribution D over $\{1, \dots, m\}$ (a probability distribution that assigns weights to training examples)

produces a hypothesis $h : \mathcal{X} \rightarrow \{-1, +1\}$ with **weighted error** (error where each training example is weighted by its distribution probability):

$$\varepsilon = \sum_{i=1}^m D(i) \cdot \mathbb{1}[h(x_i) \neq y_i] \leq \frac{1}{2} - \gamma \quad (1)$$

for some **edge** (the margin by which the learner beats random guessing) $\gamma > 0$ (independent of m).

Pause and Think

Question: Why is the threshold $1/2 - \gamma$ rather than just some constant less than $1/2$?

Hint: Consider what happens in binary classification if you guess randomly.

Answer:

Random guessing achieves 50% error. The weak learning condition requires doing *strictly better* than random, by at least γ . This $\gamma > 0$ is crucial - without it, we can't boost.

Definition: Strong Learner

In **PAC terminology** (Probably Approximately Correct, the formal learning framework from Week 2), a **strong learner** (an algorithm that can achieve arbitrarily small error with high confidence) receives iid samples from an unknown distribution, a confidence parameter $\delta \in (0, 1)$, and an accuracy parameter $\varepsilon \in (0, 1)$.

Under the relevant realizable or agnostic assumptions, it produces a hypothesis h such that, with probability at least $1 - \delta$ over the random training sample,

$$P_{x,y}[h(x) \neq y] \leq \varepsilon. \quad (2)$$

The **sample complexity** (number of training examples needed) should be polynomial in $\frac{1}{\varepsilon}$, $\frac{1}{\delta}$, and the relevant complexity parameters; the key point is that the target population error can be made **arbitrarily small**.

Connection

The PAC framework was introduced in Weeks 2-3. Strong learning satisfies the standard PAC criteria with polynomially bounded sample complexity in $\frac{1}{\varepsilon}$ and $\frac{1}{\delta}$.

The Boosting Question

Thought Experiment**The Central Question of Boosting Theory**

Given: Access to a weak learner (black box) that guarantees weighted empirical error $\leq \frac{1}{2} - \gamma$ on any distribution over the current training sample

Goal: Construct a strong learner that achieves arbitrarily small error

Allowed operations:

- Call the weak learner multiple times
- Each call can use a different distribution over training examples
- Combine the resulting weak hypotheses

Is this possible? If so, how many calls do we need?

The answer, discovered by Schapire in 1990, is **yes** in the formal PAC setting with the appropriate weak-learning oracle and generalization control. Empirically, AdaBoost drives training error down using polynomially many calls in $\frac{1}{\gamma}$ and $\log(\frac{1}{\varepsilon})$.

✓ Checkpoint

Checkpoint 2: Weak Learning Condition

Key insight: The weak learning condition ($\epsilon \leq \frac{1}{2} - \gamma$) guarantees that **every** reweighting of the training data can be learned better than random.

Test your understanding:

1. What happens if $\gamma \rightarrow 0$?
2. Why does the condition need to hold for **any** distribution D ?
3. How does this relate to the PAC model?

The distribution update is the heart of boosting. By reweighting examples, we effectively create a sequence of increasingly difficult learning problems.

The Boosting Strategy

The key insight behind boosting is a simple but powerful idea:

Key Takeaway

The Boosting Principle

1. Maintain a distribution over training examples
2. Call weak learner on current distribution
3. Increase weight on misclassified examples
4. Decrease weight on correctly classified examples
5. Repeat, forcing weak learner to focus on “hard” examples
6. Combine all weak hypotheses via weighted voting

This strategy has an elegant interpretation: we’re iteratively focusing the weak learner’s attention on the examples it finds difficult, forcing it to learn complementary patterns.

🧠 Mental Model: Adaptive Attention Mechanism

Analogy: Think of boosting like a teacher working with a student on a problem set:

- First pass: Student attempts all problems, gets some wrong
- Teacher: “Focus more on the problems you missed”
- Second pass: Student concentrates on hard problems, makes progress
- Teacher: Adjusts focus again based on new mistakes
- Repeat until student masters all problems

The **distribution** (the weight assigned to each training example) is the teacher’s attention allocation, the **weak learner** (a single hypothesis produced in one round) is the student’s attempt, and the **final classifier** (the weighted combination of all weak learners) is the combined knowledge.

The AdaBoost Algorithm

Now we present the most famous boosting algorithm: **AdaBoost** (Adaptive Boosting), developed by Freund and Schapire in 1997.

Connection

The weight update formula $D_{t+1}(i) \propto \exp(-\alpha_t y_i h_t(x_i))$ will be justified in Section 6 as gradient descent on exponential loss. For now, view it as an adaptive reweighting mechanism.

Algorithm Description

Input:

- Training set $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$ where $y_i \in \{-1, +1\}$
- Weak learning algorithm $\mathcal{A}$
- Number of rounds T

Initialize: $D_1(i) = \frac{1}{m}$ for all $i = 1, \dots, m$

For $t = 1, 2, \dots, T$:

1. **Train weak learner:**
 - Call $\mathcal{A}$ with distribution D_t
 - Obtain hypothesis $h_t : \mathcal{X} \rightarrow \{-1, +1\}$
 - Compute weighted error:

$$\varepsilon_t = \sum_{i=1}^m D_t(i) \cdot \mathbb{1}[h_t(x_i) \neq y_i] \quad (3)$$

2. **Compute hypothesis weight:**

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right) \quad (4)$$

This weight determines how much influence h_t has in the final classifier. Lower error leads to higher weight.

3. **Update distribution:**

$$D_{t+1}(i) = \frac{D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \quad (5)$$

where $Z_t = \sum_{i=1}^m D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))$ is the **normalization factor** (ensures D_{t+1} is a valid probability distribution).

Assume $0 < \varepsilon_t < \frac{1}{2}$. If $\varepsilon_t = 0$, stop with a perfect training classifier; if $\varepsilon_t \geq \frac{1}{2}$, the weak-learning condition failed (or flip the hypothesis when possible).

Output: Final hypothesis

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right) \quad (6)$$

This is a **weighted majority vote** (a linear combination where each weak learner votes with weight α_t , and the sign determines the final class).

Algorithm 1: “AdaBoost”

Pause and Think

Question: What happens to α_t when ϵ_t approaches 0?

Hint: Consider the formula $\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right)$ at the extremes.

Answer:

When $\epsilon_t \rightarrow 0$ (perfect), $\alpha_t \rightarrow \infty$ (hypothetical gets very high weight). This makes intuitive sense!

The exponential weight update is not arbitrary - it emerges naturally from the exponential loss minimization perspective we'll see later.

Understanding the Weight Updates

Let's unpack the distribution update formula:

$$D_{t+1}(i) = \frac{D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \quad (7)$$

Example: Weight Update Behavior

Consider what happens to the weight of example i :

Case 1: Correct classification ($y_i = h_{t(x_i)}$)

- Then $y_i h_{t(x_i)} = 1$
- Weight multiplier: $\exp(-\alpha_t \cdot 1) = \exp(-\alpha_t) < 1$
- Weight **decreases**

Case 2: Incorrect classification ($y_i \neq h_{t(x_i)}$)

- Then $y_i h_{t(x_i)} = -1$
- Weight multiplier: $\exp(-\alpha_t \cdot (-1)) = \exp(\alpha_t) > 1$
- Weight **increases**

Ratio: Since $\epsilon_t < \frac{1}{2}$, we have $\alpha_t > 0$. The weight of a misclassified example increases by factor

$$\exp(2\alpha_t) = \exp\left(\ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right)\right) = \frac{1 - \epsilon_t}{\epsilon_t} \quad (8)$$

For example, if $\epsilon_t = 0.3$, misclassified examples get weight multiplied by $\frac{0.7}{0.3} \approx 2.33$.

✓ Checkpoint**Checkpoint 3: Algorithm Mechanics**

Verify your understanding: Trace through AdaBoost on a tiny dataset:

- 4 examples: $(x_1, +1), (x_2, +1), (x_3, -1), (x_4, -1)$
- Round 1: h_1 misclassifies x_1 , correct on others $\rightarrow \varepsilon_1 = 0.25$
- Compute α_1 and updated weights D_2
- What happens to $D_2(1)$ compared to $D_1(1)$?

Answer: $\alpha_1 = \frac{1}{2} \ln\left(\frac{0.75}{0.25}\right) = \frac{1}{2} \ln(3) \approx 0.549$, so $D_2(1)$ increases by factor $\exp(2\alpha_1) = 3$.

Training Error Analysis

We've seen how AdaBoost works algorithmically—reweighting example learners. But this raises a crucial question: **does it actually work**? We will prove that the training error decreases, and if so, how fast?

Motivation: The weak learning assumption guarantees only that each round's error is small on average, random on the reweighted distribution. How can we convert this into a strong guarantee on the final combined classifier? The answer is one of the most elegant results in learning theory: exponential convergence.

One of the most remarkable properties of AdaBoost is that its training error decreases **exponentially fast** with the number of rounds. The next theorem analyzes only the fixed training set; population generalization is addressed later using margins.

Progress on backbone: This section answers the first key question about boosting: “Can weak learners be combined into arbitrarily strong training-set classifiers?” The answer is yes, with exponential efficiency.

Training vs Test Error

This is a training error guarantee on the fixed sample S . For generalization to unseen data, combine with margin bounds (Section 5) or PAC bounds (Week 2).

The bound is remarkable: even with a constant small edge γ , the training error decreases **exponentially** in T !

Connection

The exponential loss upper bound technique (Step 1 below) connects to the margin theory in Section 5. The margin $yF(x)$ appears naturally in the exponential loss.

The Main Theorem

Theorem: AdaBoost Training Error Bound

Fix training set $S = \{(x_i, y_i)\}_{i=1}^m$ where $y_i \in \{-1, +1\}$.

Assume the weak learner satisfies the **weak learning condition**: For each round $t = 1, \dots, T$ and distribution D_t over training indices $\{1, \dots, m\}$, the weak learner returns $h_t : \mathcal{X} \rightarrow \{-1, +1\}$ with weighted training error:

$$\varepsilon_t = \sum_{i=1}^m D_t(i) \cdot \mathbb{1}_{h_t(x_i) \neq y_i} \leq \frac{1}{2} - \gamma_t \quad (9)$$

where $\gamma_t > 0$ is the edge in round t .

Then the empirical training error (0-1 loss) of the final ensemble $H(x) = \text{sgn}(F_T(x))$ satisfies:

$$\frac{1}{m} \sum_{i=1}^m \mathbb{1}_{H(x_i) \neq y_i} \leq \prod_{t=1}^T 2\sqrt{\varepsilon_t(1 - \varepsilon_t)} = \prod_{t=1}^T \sqrt{1 - 4\gamma_t^2} \quad (10)$$

Proof Strategy

The proof uses a beautiful technique: upper bound the discrete 0-1 loss by a smooth exponential loss, then track how this exponential loss evolves through the boosting rounds.

Proof Template: Exponential Loss Telescoping

Pattern: Upper bound discrete loss $\rightarrow$ Track smooth surrogate $\rightarrow$ Relate to algorithm updates

Step 1: Show $\mathbb{1}[yF(x) \leq 0] \leq \exp(-yF(x))$ (**convex relaxation**, replacing a non-convex 0-1 loss with a convex exponential upper bound) **Step 2:** Relate $\sum_i \exp(-y_i F(x_i))$ to normalization factors Z_t (**invariant**, a quantity that has a simple relationship across iterations) **Step 3:** Bound each Z_t using weak learning condition (**edge analysis**, exploiting the guaranteed accuracy advantage) **Step 4:** Product of bounds gives exponential convergence

This template appears in many learning algorithms that minimize **convex surrogates** (smooth convex functions that upper bound the non-convex 0-1 loss) for 0-1 loss.

Proof. Step 1: Upper bound via exponential loss

For any example (x_i, y_i) and combined classifier $F(x) = \sum_{t=1}^T \alpha_t h_t(x)$

$$\mathbb{1}_{y_i F(x_i) \leq 0} \leq \exp(-y_i F(x_i))$$

This exponential upper bound exploits **convexity** (Week 1): The exponential function is convex and lies above the step function at the critical point 0.

Case 1: Misclassified ($y_i F(x_i) \leq 0$)

- Left side: $\mathbb{1}_{\dots} = 1$
- Right side: $\exp(-y_i F(x_i)) \geq \exp(0) = 1$
- Thus: $1 \leq \exp(-y_i F(x_i)) \checkmark$

Case 2: Correctly classified ($y_i F(x_i) > 0$)

- Left side: $\mathbb{1}_{\dots} = 0$
- Right side: $\exp(-y_i F(x_i)) > 0$
- Thus: $0 \leq \exp(-y_i F(x_i)) \checkmark$

In both cases, the inequality holds. This is a **convex relaxation**: we replace the non-convex 0-1 loss with a smooth convex exponential upper bound.

Summing over all training examples:

$$\text{Training error} = \frac{1}{m} \sum_{i=1}^m \mathbb{1}_{H(x_i) \neq y_i} \leq \frac{1}{m} \sum_{i=1}^m \exp(-y_i F(x_i)) \quad (12)$$

Step 2: Relate exponential loss to normalization factors

We claim that:

$$\frac{1}{m} \sum_{i=1}^m \exp(-y_i F(x_i)) = \prod_{t=1}^T Z_t \quad (13)$$

where Z_t is the normalization factor from round t .

Proof by induction on the invariant:

We prove by induction that for all rounds $t = 1, \dots, T$:

$$D_t(i) = \frac{\exp(-y_i F_{t-1}(x_i))}{m \prod_{s=1}^{t-1} Z_s} \quad (14)$$

where $F_0(x) = 0$ (empty sum) and the empty product equals 1.

Base case ($t = 1$): Initially $D_1(i) = \frac{1}{m}$ (uniform distribution). Since $F_0(x_i) = 0$:

$$\frac{\exp(-y_i \cdot 0)}{m \cdot 1} = \frac{1}{m} \quad (15)$$

✓

Inductive step: Assume the invariant holds for round t . The AdaBoost update rule is:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \quad (16)$$

Substituting the inductive hypothesis:

$$\begin{aligned} D_{t+1}(i) &= \frac{1}{Z_t} \cdot \frac{\exp(-y_i F_{t-1}(x_i))}{m \prod_{s=1}^{t-1} Z_s} \cdot \exp(-\alpha_t y_i h_t(x_i)) \\ &= \frac{\exp(-y_i F_{t-1}(x_i)) \exp(-\alpha_t y_i h_t(x_i))}{m \prod_{s=1}^t Z_s} \\ &= \frac{\exp(-y_i [F_{t-1}(x_i) + \alpha_t h_t(x_i)])}{m \prod_{s=1}^t Z_s} \\ &= \frac{\exp(-y_i F_t(x_i))}{m \prod_{s=1}^t Z_s} \end{aligned} \quad (17)$$

✓

Now, summing the invariant over all training indices at time $T + 1$:

$$1 = \sum_{i=1}^m D_{T+1}(i) = \frac{1}{m \prod_{t=1}^T Z_t} \sum_{i=1}^m \exp(-y_i F_T(x_i)) \quad (18)$$

Rearranging:

$$\frac{1}{m} \sum_{i=1}^m \exp(-y_i F_T(x_i)) = \prod_{t=1}^T Z_t \quad (19)$$

Step 3: Bound the normalization factor

Recall that:

$$Z_t = \sum_{i=1}^m D_{t(i)} \exp(-\alpha_t y_i h_t(x_i)) \quad (20)$$

Split the sum into correctly and incorrectly classified examples:

$$\begin{aligned} Z_t &= \sum_{i: h_t(x_i) = y_i} D_{t(i)} \exp(-\alpha_t) + \sum_{i: h_t(x_i) \neq y_i} D_{t(i)} \exp(\alpha_t) \\ &= (1 - \varepsilon_t) \exp(-\alpha_t) + \varepsilon_t \exp(\alpha_t) \end{aligned} \quad (21)$$

Now substitute $\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$:

$$\exp(\alpha_t) = \sqrt{\frac{1 - \varepsilon_t}{\varepsilon_t}}, \quad \exp(-\alpha_t) = \sqrt{\frac{\varepsilon_t}{1 - \varepsilon_t}} \quad (22)$$

Therefore:

$$\begin{aligned} Z_t &= (1 - \varepsilon_t) \sqrt{\frac{\varepsilon_t}{1 - \varepsilon_t}} + \varepsilon_t \sqrt{\frac{1 - \varepsilon_t}{\varepsilon_t}} \\ &= \sqrt{(1 - \varepsilon_t)\varepsilon_t} + \sqrt{\varepsilon_t(1 - \varepsilon_t)} \\ &= 2\sqrt{\varepsilon_t(1 - \varepsilon_t)} \end{aligned} \quad (23)$$

Step 4: Combine the bounds

$$\text{Training error} \leq \prod_{t=1}^T Z_t = \prod_{t=1}^T 2\sqrt{\varepsilon_t(1 - \varepsilon_t)} \quad (24)$$

Using $\varepsilon_t = \frac{1}{2} - \gamma_t$:

$$\sqrt{\varepsilon_t(1 - \varepsilon_t)} = \sqrt{\left(\frac{1}{2} - \gamma_t\right)\left(\frac{1}{2} + \gamma_t\right)} = \sqrt{\frac{1}{4} - \gamma_t^2} = \frac{1}{2}\sqrt{1 - 4\gamma_t^2} \quad (25)$$

Therefore:

$$\text{Training error} \leq \prod_{t=1}^T \sqrt{1 - 4\gamma_t^2} \quad (26)$$

■

■

Pause and Think

Question: Why did we choose $\alpha_t = (1/2)\ln((1-\varepsilon_t)/\varepsilon_t)$? Is this arbitrary?

Hint: Look at Step 3 of the proof. What value of α_t minimizes Z_t ?

Answer:

The choice of α_t is precisely the value that minimizes Z_t ! Taking the derivative of Z_t with respect to α and setting it to zero yields exactly this formula. This is optimal for minimizing the training error bound.

✓ Checkpoint

Checkpoint 4: Proof Understanding

Critical insights from the proof:

1. Exponential loss provides a smooth, tractable upper bound
2. The invariant $D_{t+1}(i) \propto \exp(-y_i F_t(x_i))$ connects distributions to loss
3. The α_t formula is optimal—it minimizes Z_t
4. The product structure $\prod_t Z_t$ enables exponential convergence

Self-check: Why is Step 2's invariant true? Verify it for $t = 1$ explicitly.

Exponential Convergence

The real power of the theorem becomes clear when the weak learner:

Theorem: Exponential Convergence with Constant Edge

Assume the weak learner satisfies the **constant edge condition**

For all rounds $t = 1, \dots, T$ and all distributions D_t generated by Adaboost, the weak learner returns h_t with weighted training error:

$$\varepsilon_t = \sum_{i=1}^m D_t(i) \cdot \mathbb{1}_{h_t(x_i) \neq y_i} \leq \frac{1}{2} - \gamma \quad (27)$$

for some fixed $\gamma > 0$ (the constant edge).

Then the training error converges exponentially:

$$\text{Training error} \leq \exp(-2\gamma^2 T) \quad (28)$$

In particular, to achieve training error $\leq \delta$, it suffices to run for

$$T = O\left(\left(\frac{1}{\gamma^2}\right) \log\left(\frac{1}{\delta}\right)\right) \quad (29)$$

rounds.

Constant edge assumption

The constant edge condition is strong—it requires the weak learner to succeed on EVERY reweighted distribution D_t , no matter how the algorithm adapts.

In practice, the edge may vary across rounds, but the bound still provides useful intuition about the exponential convergence rate.

Proof. Using the inequality $1 - x \leq \exp(-x)$ for $x \geq 0$:

$$\sqrt{1 - 4\gamma^2} \leq \exp(-2\gamma^2) \quad (30)$$

Therefore:

$$\text{Training error} \leq \prod_{t=1}^T \sqrt{1 - 4\gamma^2} \leq \exp(-2\gamma^2 T) \quad (31)$$

To achieve training error $\leq \delta$, we need:

$$\exp(-2\gamma^2 T) \leq \delta \quad (32)$$

$$T \geq \frac{\ln\left(\frac{1}{\delta}\right)}{2\gamma^2} \quad (33)$$

■

■

Example: Concrete Numbers

Suppose the weak learner achieves 51% accuracy ($\gamma = 0.01$). How many rounds are needed for 1% training error?

$$T \geq \frac{\ln(100)}{2 \cdot 0.01^2} = \frac{4.605}{0.0002} \approx 23,025 \quad (34)$$

Even with such a weak learner, about 23,000 rounds suffice to drive training error below 1%!

With a stronger weak learner achieving 60% accuracy ($\gamma = 0.1$):

$$T \geq \frac{\ln(100)}{2 \cdot 0.1^2} = \frac{4.605}{0.02} \approx 230 \quad (35)$$

Only 230 rounds needed!

Key Takeaway

AdaBoost converts weak learners into a strong **training-set** classifier with **exponential** efficiency. The training error decreases as $\exp(-2\gamma^2 T)$, where γ is the edge of the weak learner.

Connection

This theorem is the empirical core of boosting: given a weak learner with edge γ on every reweighted training distribution, we can drive training error below ε in $O\left(\left(\frac{1}{\gamma^2}\right) \log\left(\frac{1}{\varepsilon}\right)\right)$ rounds.

The full weak-PAC-to-strong-PAC theorem adds sample-complexity and generalization arguments on top of this training-error analysis.

The Margin Explanation

Motivation: AdaBoost drives training error to zero rapidly—but why does it generalize well to unseen data? The answer lies in **margins**. Even after perfect training accuracy, AdaBoost continues to increase the margin (confidence) of correct predictions, which improves generalization.

← From Week 2: PAC Learning & VC Dimension

Week 2 showed that generalization depends on hypothesis class complexity (VC dimension). But AdaBoost combines many weak learners—doesn't this increase complexity? The margin theory resolves this paradox.

Classification Margin

Definition: Classification Margin

For a training example (x_i, y_i) and ensemble classifier $F(x) = \sum_{t=1}^T \alpha_t h_t(x)$, the **unnormalized margin** is:

$$\text{margin}_{\text{unnorm}}(x_i, y_i) = y_i F(x_i) = y_i \sum_{t=1}^T \alpha_t h_t(x_i) \quad (36)$$

The **normalized margin** (scaled to $[-1, 1]$) is:

$$\text{margin}(x_i, y_i) = \frac{y_i F(x_i)}{\|\alpha\|_1} = \frac{y_i \sum_{t=1}^T \alpha_t h_t(x_i)}{\sum_{t=1}^T |\alpha_t|} \quad (37)$$

Interpretation:

- Margin > 0 : Correct prediction
- Margin ≈ 1 : High confidence correct prediction
- Margin ≈ 0 : Near decision boundary (low confidence)
- Margin < 0 : Misclassification

Example: Margin Intuition

Consider ensemble $F(x) = 0.5h_1(x) + 0.3h_2(x) + 0.2h_3(x)$ with $\|\alpha\|_1 = 1$.

For a positive example ($y = +1$):

- If $h_1(x) = +1, h_2(x) = +1, h_3(x) = +1$: unanimous agreement

$$\text{margin} = \frac{+1 \cdot (0.5 + 0.3 + 0.2)}{1} = +1 \quad (38)$$

(maximum confidence)

- If $h_1(x) = +1, h_2(x) = +1, h_3(x) = -1$: majority vote

$$\text{margin} = \frac{+1 \cdot (0.5 + 0.3 - 0.2)}{1} = +0.6 \quad (39)$$

(moderate confidence)

- If $h_1(x) = +1, h_2(x) = -1, h_3(x) = -1$: minority correct

$$\text{margin} = \frac{+1 \cdot (0.5 - 0.3 - 0.2)}{1} = 0 \quad (40)$$

(decision boundary)

Large margins indicate robust, confident predictions.

Key Takeaway

The margin measures **confidence** of prediction, not just correctness. AdaBoost naturally maximizes margins by continuing to adapt weights even after training error reaches zero.

Margin-Based Generalization Bound

The power of margins: they provide generalization bounds that depend on margin size, not directly on the number of weak learners T .

Theorem: Margin Bound for Ensembles

Let $\mathcal{H}$ be the base hypothesis class with VC dimension d . Fix margin threshold $\theta > 0$.

With probability $\geq 1 - \delta$ over training samples $S \sim \mathcal{D}^m$, for any ensemble $F = \sum_{t=1}^T \alpha_t h_t$ where $h_t \in \mathcal{H}$:

$$R_{\mathcal{D}}(\text{sgn}(F)) \leq \frac{1}{m} |\{i : \text{margin}(x_i, y_i) < \theta\}| + O\left(\sqrt{\frac{d \log^2(\frac{m}{\theta}) + \log(\frac{1}{\delta})}{m}}\right) \quad (41)$$

where $|\{\dots\}|$ counts training examples with margin below θ .

Proof. Sketch: The key insight is that the margin-based hypothesis class

$$\mathcal{H}_{\text{margin}} = \{\text{sgn}(F) : \|\alpha\|_1 \leq 1, \text{margin} \geq \theta\}$$

has VC dimension $O(d \log(\frac{1}{\theta}))$ rather than depending on T .

Apply VC bounds (Week 2) to this margin-based class. The first term is on training set; the second term is the VC convergence rate.

Why doesn't T appear?

The bound depends on VC dimension d of base learners and margin θ , but NOT on the number of rounds T .

This explains why AdaBoost can run for many rounds without over-

Example: Margin vs Ensemble Size

Consider two ensembles with base class VC dimension $d = 10$:

Ensemble A: $T = 10$ rounds, minimum margin $\theta = 0.5$

- Complexity penalty: $O\left(\sqrt{\frac{10 \log^2(\frac{m}{0.5})}{m}}\right)$

Ensemble B: $T = 100$ rounds, minimum margin $\theta = 0.8$

- Complexity penalty: $O\left(\sqrt{\frac{10 \log^2(\frac{m}{0.8})}{m}}\right)$

Despite $10\times$ more weak learners, Ensemble B may generalize better because the larger margin $\theta = 0.8$ dominates the $\log(\frac{1}{\theta})$ factor.

Connection to Exponential Loss

Why does AdaBoost naturally maximize margins? The exponential loss provides the answer.

Key Takeaway

Exponential loss is a margin-maximizing surrogate

Minimizing $\sum_i \exp(-y_i F(x_i))$ drives $y_i F(x_i)$ (the unnormalized margin) toward $+\infty$ for all training examples. This continues even after training error reaches zero.

Connection

The relationship between exponential loss and margins:

$$\mathcal{L}_{\text{exp}(F)} = \sum_{i=1}^m \exp(-y_i F(x_i)) \quad (43)$$

- When margin $y_i F(x_i)$ is large and positive: loss ≈ 0 (already confident)
- When margin $y_i F(x_i)$ is small: loss is large (drives further optimization)
- When margin $y_i F(x_i)$ is negative: loss is VERY large (severe penalty for mistakes)

AdaBoost's weight updates implicitly minimize this exponential loss, which implicitly maximizes margins.

→ Week 11 Preview: Neural Networks & Deep Learning

The margin interpretation generalizes to deep learning: neural networks trained with cross-entropy loss (closely related to exponential loss) also benefit from large margins. This connects classical boosting theory to modern deep learning.

Loss Minimization Perspective

Motivation: We've seen AdaBoost as an adaptive reweighting algorithm. But there's a deeper view: AdaBoost performs **coordinate descent** on exponential loss. This connects boosting to optimization theory and reveals why the algorithm works.

← From Week 1: Convex Optimization

Week 1 introduced gradient descent for smooth convex functions. AdaBoost performs a variant called **coordinate descent**: optimize one coordinate (weak learner) at a time while holding others fixed.

Exponential Loss as Objective

Definition: Exponential Loss Function

For ensemble $F(x) = \sum_{t=1}^T \alpha_t h_t(x)$ and training set $S = \{(x_i, y_i)\}_{i=1}^m$:

$$\mathcal{L}_{\text{exp}}(F) = \sum_{i=1}^m \exp(-y_i F(x_i)) \quad (44)$$

This is a **convex** function of the coefficients $\{\alpha_t\}$ (Week 1 convexity).

Key Takeaway

AdaBoost can be viewed as **greedy coordinate descent** on exponential loss:

1. Fix current ensemble F_{t-1}
2. Choose next weak learner h_t to maximize loss reduction
3. Choose weight α_t via line search to minimize loss
4. Update: $F_t = F_{t-1} + \alpha_t h_t$

Coordinate Descent Interpretation

Theorem: AdaBoost as Coordinate Descent

Each AdaBoost iteration performs the following optimization:

Step 1 (Weak learner selection): Choose h_t to maximize weight

$$h_t = \operatorname{argmax}_{h \in \mathcal{H}} \sum_{i=1}^m D_{t(i)} \cdot y_i h(x_i)$$

This is equivalent to minimizing the loss if h is added to the current ensemble.

Step 2 (Weight selection): Choose α_t to minimize loss via line search:

$$\alpha_t = \operatorname{argmin}_{\alpha \geq 0} \sum_{i=1}^m \exp\left(-y_i [F_{t-1}(x_i) + \alpha h_{t(x_i)}]\right) \quad (46)$$

Solving this optimization yields:

$$\alpha_t = \frac{1}{2} \log \frac{1 - \varepsilon_t}{\varepsilon_t} \quad (47)$$

where ε_t is the weighted error of h_t under D_t .

Why coordinate descent?

Optimizing over all possible ensembles $\{\alpha_t, h_t\}$ simultaneously is computationally intractable (exponentially many weak learners).

Coordinate descent (optimize one weak learner at a time) is tractable and provably converges for convex losses like exponential loss.

Proof. Step 1 verification: The loss gradient at F_{t-1} is:

$$\frac{\partial \mathcal{L}_{\text{exp}}}{\partial F(x_i)} = -y_i \exp(-y_i F_{t-1}(x_i)) \quad (48)$$

To decrease loss, we want h_t aligned with the negative gradient:

$$\max_h \sum_i -y_i \exp(-y_i F_{t-1}(x_i)) \cdot h(x_i) = \max_h \sum_i D_{t(i)} \cdot y_i h(x_i) \quad (49)$$

where $D_{t(i)} \propto \exp(-y_i F_{t-1}(x_i))$ (the AdaBoost distribution!).

Step 2 verification: Fixing h_t , we minimize:

$$\mathcal{L}(\alpha) = \sum_i \exp\left(-y_i [F_{t-1}(x_i) + \alpha h_{t(x_i)}]\right) \quad (50)$$

Taking derivative with respect to α :

$$\frac{d\mathcal{L}}{d\alpha} = - \sum_i y_i h_{t(x_i)} \exp\left(-y_i [F_{t-1}(x_i) + \alpha h_{t(x_i)}]\right) \quad (51)$$

Setting to zero and solving (after algebraic manipulation):

$$\alpha_t = \frac{1}{2} \log \frac{1 - \varepsilon_t}{\varepsilon_t} \quad (52)$$

This matches the AdaBoost formula exactly! ■

Why Exponential Loss?

Exponential loss has special properties that make boosting work:

Key Takeaway

Advantages of exponential loss:

1. **Convex:** Enables optimization theory (no local minima)
2. **Differentiable:** Allows gradient-based interpretation
3. **Margin-maximizing:** Drives margins toward infinity (generalization)
4. **Analytically tractable:** Closed-form solution for α_t

Disadvantage:

- Sensitive to outliers/noise (grows exponentially for large negative margins)

Loss Functions for Classification

Loss	Formula	Convex?	Properties
0-1 loss	$\mathbb{1}_{yF(x) \leq 0}$	No	What we care about, but non-convex
Exponential	$\exp(-yF(x))$	Yes	AdaBoost, margin-maximizing, outlier-sensitive
Logistic	$\log(1 + \exp(-yF(x)))$	Yes	LogitBoost, more robust to outliers
Hinge	$\max(0, 1 - yF(x))$	Yes	SVM, sparsity (only support vectors matter)

Connection

The coordinate descent view unifies boosting with other learning algorithms:

- **Gradient boosting:** Replace coordinate descent with full gradient descent
- **LogitBoost:** Replace exponential loss with logistic loss
- **AdaBoost:** Special case with exponential loss and discrete weak learners

All share the same optimization framework: minimize a convex loss surrogate via greedy function approximation.

